

Artificial Intelligence - Deep Learning fundamentals

Master in Artificial Intelligence & Computational Engineering
Luis Montesano



Universidad
Zaragoza

In these slides

- **Some important DNN *ideas* and *ingredients***
 - forward-backward pass
 - back propagation
 - Parameters (model) and hyperparameters (config)
 - Bias - Variance. Regularization
 - Initialisation and performance

General formulation

$$\mathbf{h}_1 = \mathbf{a}[\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}]$$

$$\mathbf{h}_2 = \mathbf{a}[\boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1 \mathbf{h}_1]$$

$$\mathbf{h}_3 = \mathbf{a}[\boldsymbol{\beta}_2 + \boldsymbol{\Omega}_2 \mathbf{h}_2]$$

$$\vdots$$

$$\mathbf{h}_K = \mathbf{a}[\boldsymbol{\beta}_{K-1} + \boldsymbol{\Omega}_{K-1} \mathbf{h}_{K-1}]$$

$$\mathbf{y} = \boldsymbol{\beta}_K + \boldsymbol{\Omega}_K \mathbf{h}_K,$$

$$\mathbf{y} = \boldsymbol{\beta}_K + \boldsymbol{\Omega}_K \mathbf{a} [\boldsymbol{\beta}_{K-1} + \boldsymbol{\Omega}_{K-1} \mathbf{a} [\dots \boldsymbol{\beta}_2 + \boldsymbol{\Omega}_2 \mathbf{a} [\boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1 \mathbf{a} [\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}]] \dots]]]$$

- Bias vectors and weight vectors

How to compute the gradient

Loss: sum of individual terms:

$$L[\phi] = \sum_{i=1}^I \ell_i = \sum_{i=1}^I l[f[\mathbf{x}_i, \phi], y_i]$$

SGD Algorithm:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

Parameters:

$$\phi = \{\beta_0, \Omega_0, \beta_1, \Omega_1, \beta_2, \Omega_2, \beta_3, \Omega_3\}$$

Need to compute gradients

$$\frac{\partial \ell_i}{\partial \beta_k} \quad \text{and} \quad \frac{\partial \ell_i}{\partial \Omega_k}$$

Why is this an issue?

Why is this an issue?

- We need the derivative of
 - Every parameter
 - For every training example in the batch
 - For every iteration of SGD
- Number of parameters
 - Toy example used in the first lesson on DN?
 - LLMs? Computer vision models?

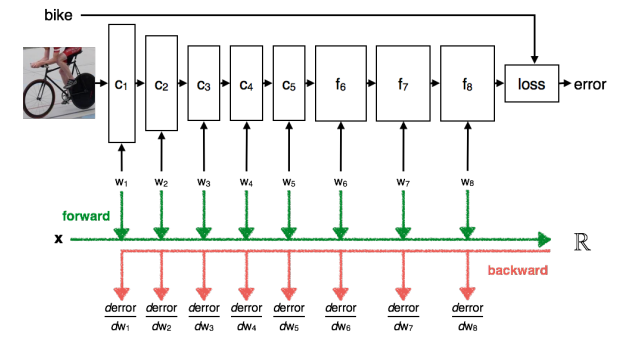
$$\begin{aligned} y' = & \phi'_0 + \phi'_1 a[\psi_{10} + \psi_{11} a[\theta_{10} + \theta_{11} x] + \psi_{12} a[\theta_{20} + \theta_{21} x] + \psi_{13} a[\theta_{30} + \theta_{31} x]] \\ & + \phi'_2 a[\psi_{20} + \psi_{21} a[\theta_{10} + \theta_{11} x] + \psi_{22} a[\theta_{20} + \theta_{21} x] + \psi_{23} a[\theta_{30} + \theta_{31} x]] \\ & + \phi'_3 a[\psi_{30} + \psi_{31} a[\theta_{10} + \theta_{11} x] + \psi_{32} a[\theta_{20} + \theta_{21} x] + \psi_{33} a[\theta_{30} + \theta_{31} x]] \end{aligned}$$

Backpropagation (intuition)

(Rumelhart, Hinton, Williams 1986)

- Forward pass
 - Perform inference from the inputs to the final function and error; and store activations at each node
- Backward pass
 - Compute derivatives at each node using chain rule
 - Trace the error backward through the chain of cause and effect, using local derivatives to assign precise "blame" ($\partial w \partial \text{Loss}$) to every parameter.

Deep Neural Network



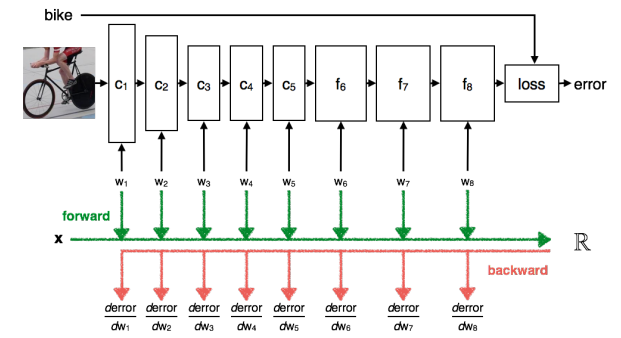
Optimisation/Training - Toy example of back propagation

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$

$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$$

How do we do this?
*Analytically would be just
too long for “deep” sizes ...*

Deep Neural Network



Optimisation/Training - Toy example of back propagation

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$

$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$$

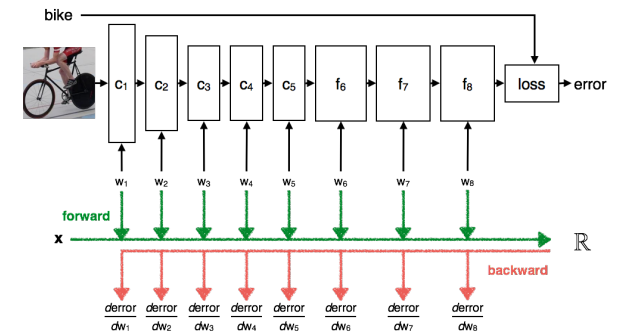
How do we do this?

Analytically would be just too long for “deep” sizes ...

L is not directly a function of all W, but is a function of a function of a function, etc

—> **CHAIN RULE**

Deep Neural Network



Optimisation/Training - Toy example of back propagation



How do we do this?
Analytically would be just too long for “deep” sizes ...

$$f(x, y, z) = (x + y)z$$

Chain Rule

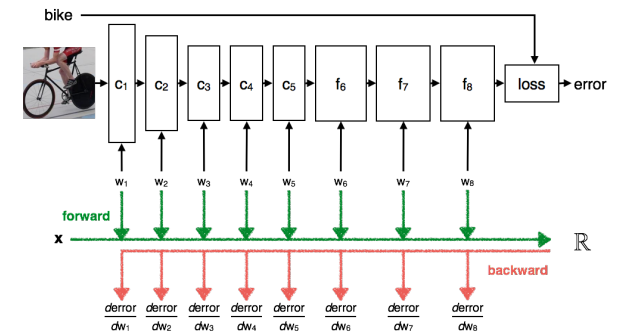
$$f = q \cdot z$$

$$df/dq = z$$

...

$$df/dx = df/dq * dq/dx$$

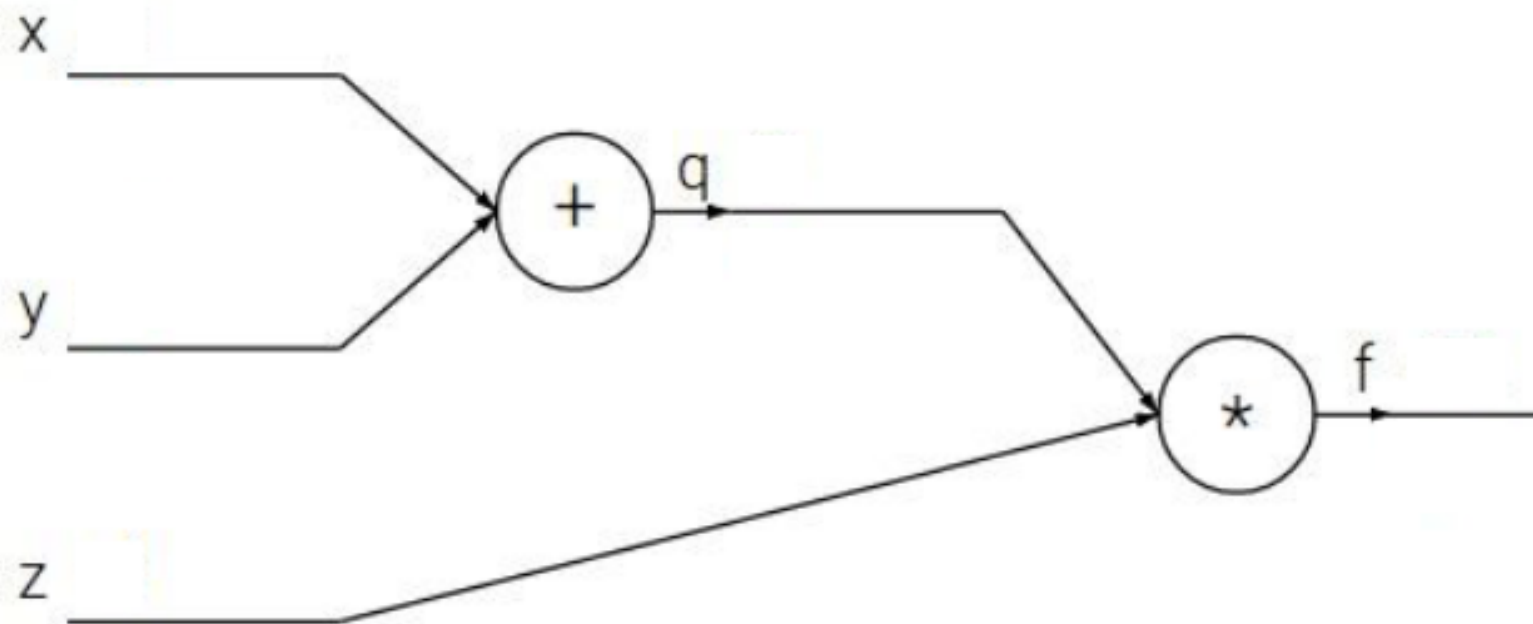
Deep Neural Network



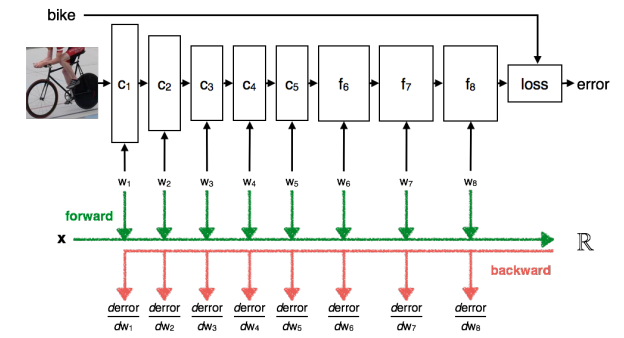
Optimisation/Training - Toy example of back propagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



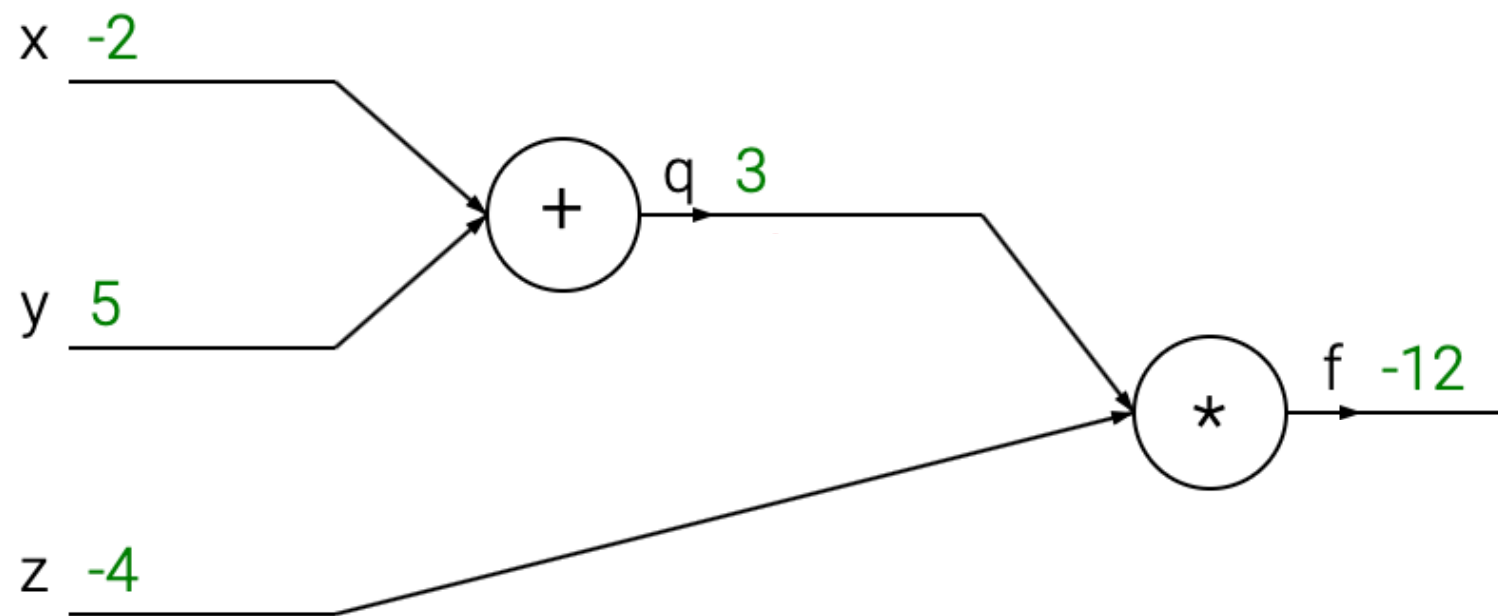
Deep Learning



Optimisation/Training - Toy example of back propagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$



Want: $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$

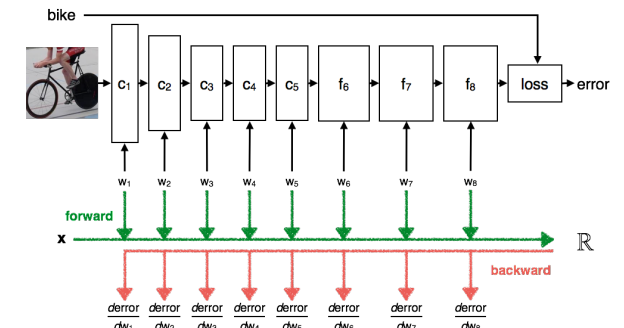
Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream
gradient

Local
gradient

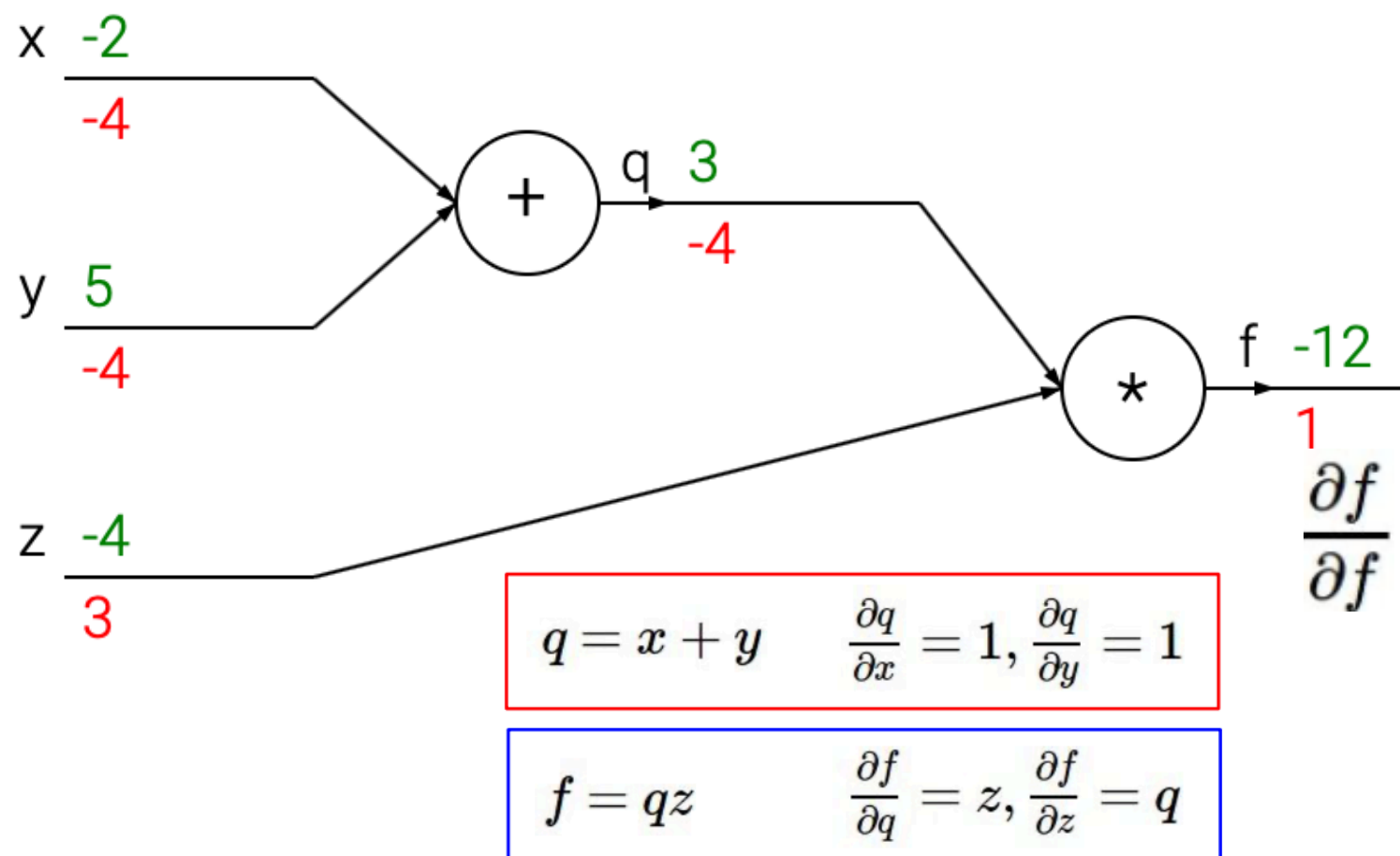
Deep Learning



Optimisation/Training - Toy example of back propagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

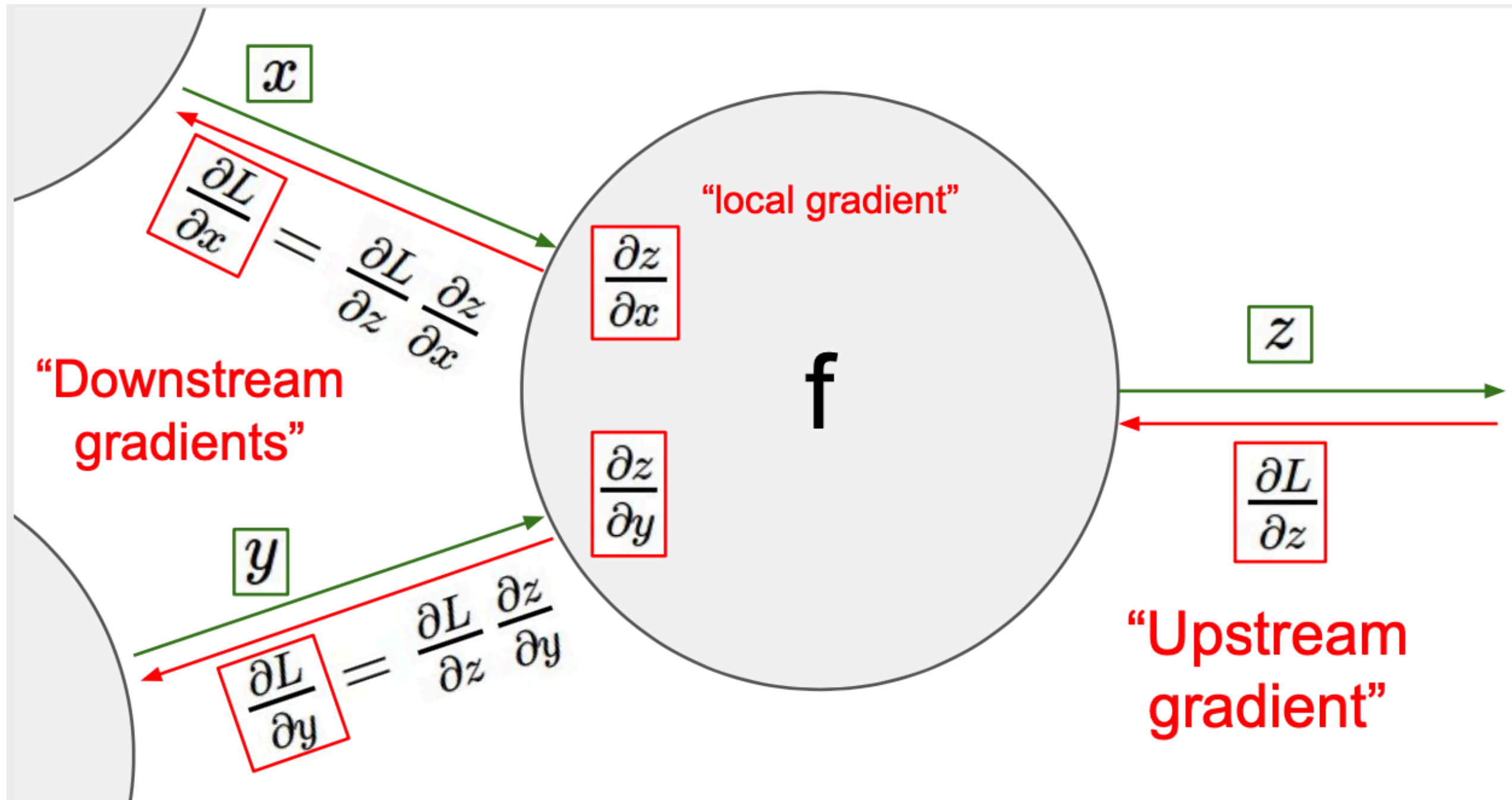
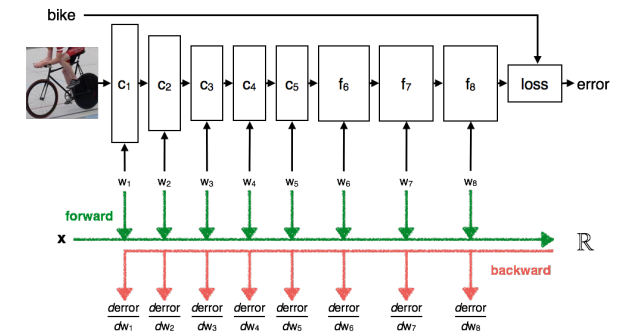
Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

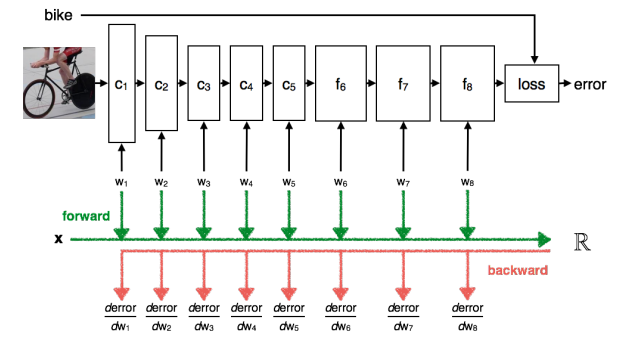
Upstream
gradient

Local
gradient

Deep Neural Network



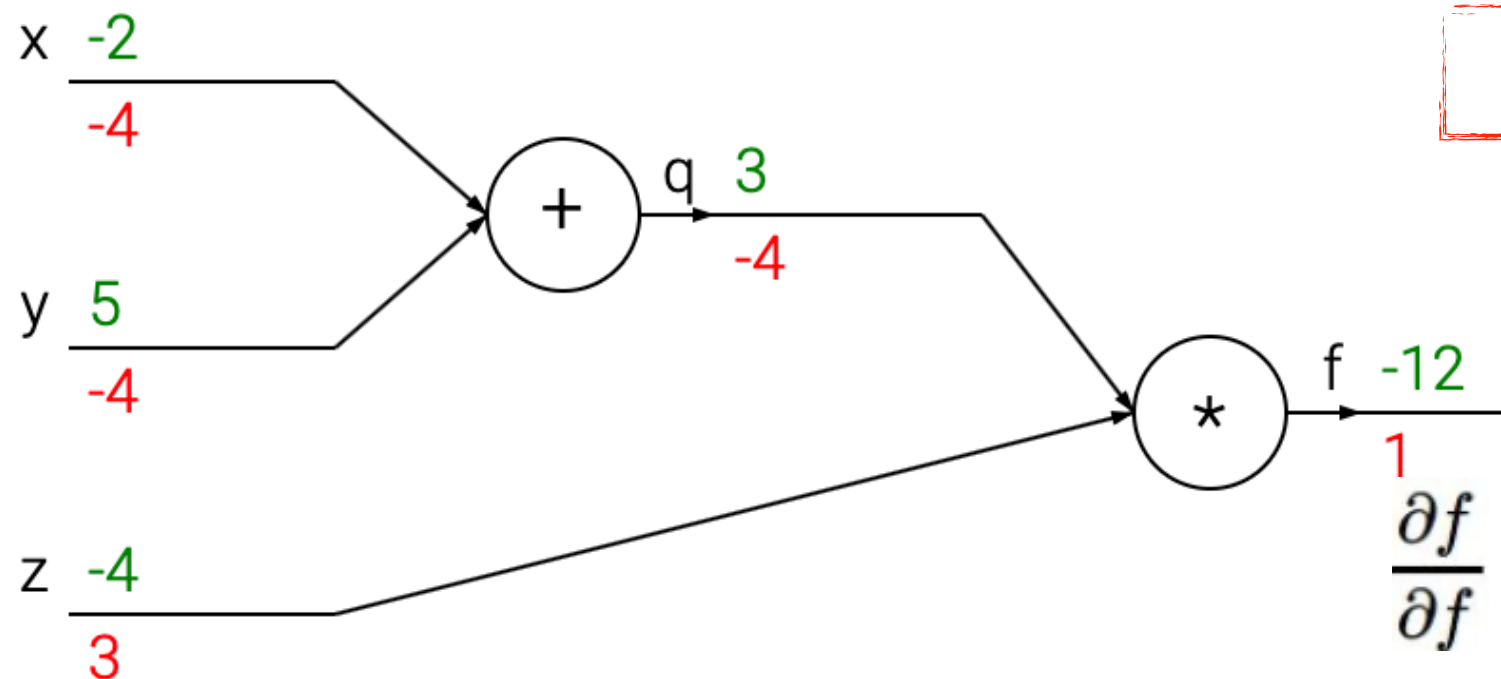
Deep Learning



Optimisation/Training - Toy example of back propagation

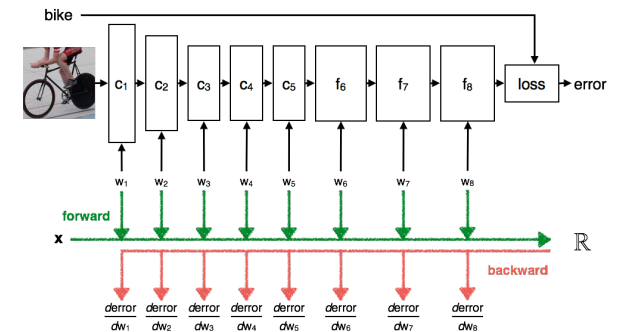
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$



And what does this mean?

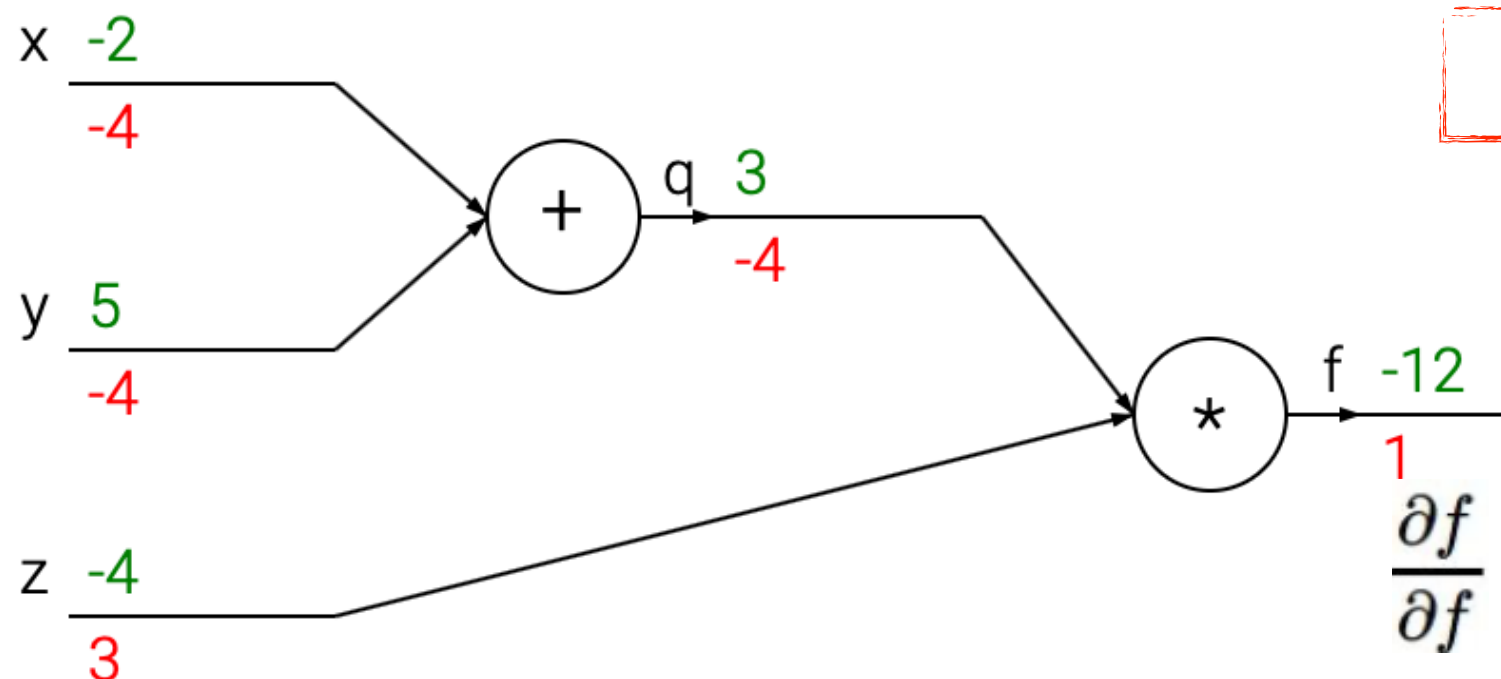
Deep Learning



Optimisation/Training - Toy example of back propagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



And what does this mean?

Weights update:

$$w = w - lr * dw$$

$$W_{1,} = W_{1,} - lr \frac{\partial L}{\partial W_{1,}} ; \dots$$

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9      h = 1 / (1 + np.exp(-x.dot(w1)))
10     y_pred = h.dot(w2)
11     loss = np.square(y_pred - y).sum()
12     print(t, loss)
13
14     grad_y_pred = 2.0 * (y_pred - y)
15     grad_w2 = h.T.dot(grad_y_pred)
16     grad_h = grad_y_pred.dot(w2.T)
17     grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19     w1 -= 1e-4 * grad_w1
20     w2 -= 1e-4 * grad_w2
```

Define the network

Forward pass

Calculate the analytical gradients

Gradient descent

$$\text{Hidden Activations: } h = \sigma(XW_1) = \frac{1}{1 + e^{-XW_1}}$$

$$\text{Predictions: } \hat{Y} = hW_2 = \sigma(XW_1)W_2$$

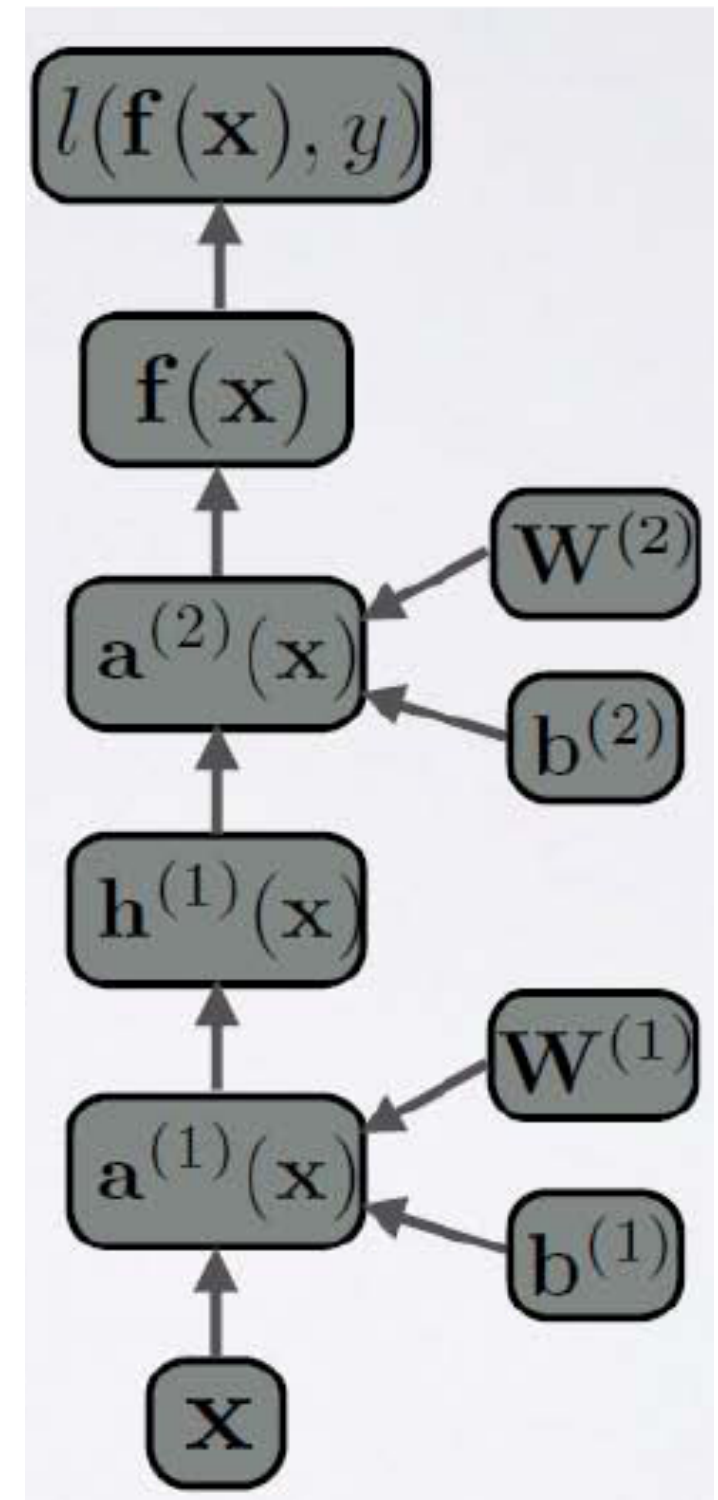
Backprop details

why we don't need the explicit gradients

- Do you know this?
- Not really necessary at first, but provides a deep understanding of how the training actually works
- Might be necessary in some specific situations if you need to write low level code / optimise

Back-propagation computational flow

- Forward propagation can be seen as an acyclic flow graph.
- Each object (box) has:
 - A forward function of its output depending on its input.
 - A gradient function of its output depending on its input that is computed backwards.
- Software implementations are usually designed according to this model.



Deep Neural Network



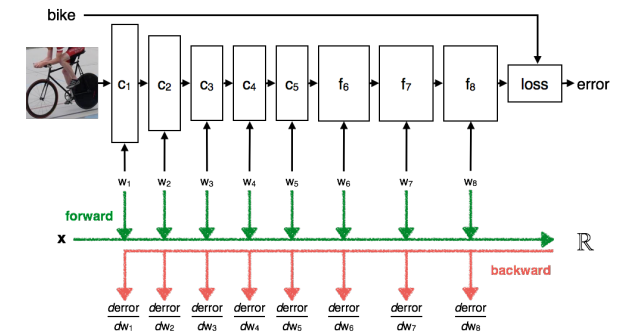
Optimisation/Training:

start with random weights & iteratively refine to get lower loss

```
while True:
    weights_grad = evaluate_gradient(loss_fun, data, weights)
    weights += - step_size * weights_grad # perform parameter update
```

- **miniB SGD**: gradient over batches. improve performance
- **backpropagation**: gradient analytically using *chain rule*
 - gates communicate to each other what they need in order to increase the final output (score).
 - allows to optimize relatively arbitrary loss functions (to define all kinds of NN, e.g. CNN)

Deep Neural Network



Optimisation/Training in practice:

- Network = operations *chained* together, each one has a simple derivative
- Graph structure. The nodes implement the **forward()** / **backward()** API

- **forward pass function** (local score from its *forward* input)
= **score** (using layer weights)



$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$

$$f = q z$$

- **backward pass function** (local gradient from its *backwards* input)
= $d_error / d_layer\text{-}weights$



$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$$

$$df/dz = q$$

- **update** $W_1 = W_1 - lr \frac{\partial L}{\partial W_1} ; \dots$

CODE

- No need for writing gradients
- Most of the time autograd computes them for you

Python

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)

# Forward pass
u = x * y          # u = 6.0 (grad_fn=<MulBackward0>)
z = u + b          # z = 7.0 (grad_fn=<AddBackward0>)

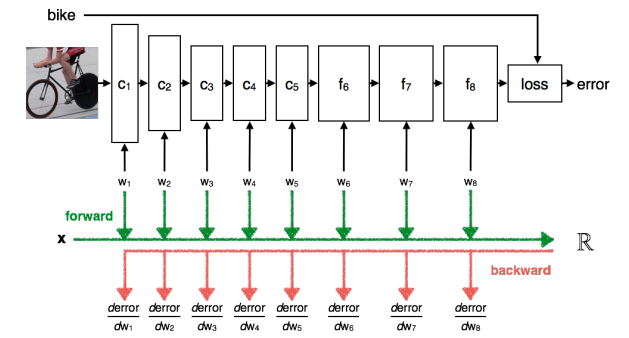
# Backward pass
z.backward()
```

What Actually Gets Stored at Each Node During the Forward Pass

When you execute an operation, autograd attaches a node to the computational graph that stores only two things:

1. **The VJP function definition** (e.g., `MulBackward`, `AddBackward`, `ReluBackward`).
2. **Saved tensors (Context)** needed to evaluate that specific function later (e.g., saving x for $y = x^2$, or saving W for $Y = XW$).

Deep Neural Network



Optimization/Training. Common (and necessary) practices:

LAB 3

- **Normalize** input data



- **Initialize** weights. Not zero but small random numbers.

- **Batch normalization:** normalizes a layer's inputs with mean and std of current mini-batch and re-scales and shifts using two learnable parameters.

- Strong **regularization** - avoid overfitting (regularization strength, dropout, augmentation, ...)



- Different **loss functions** depending on the task

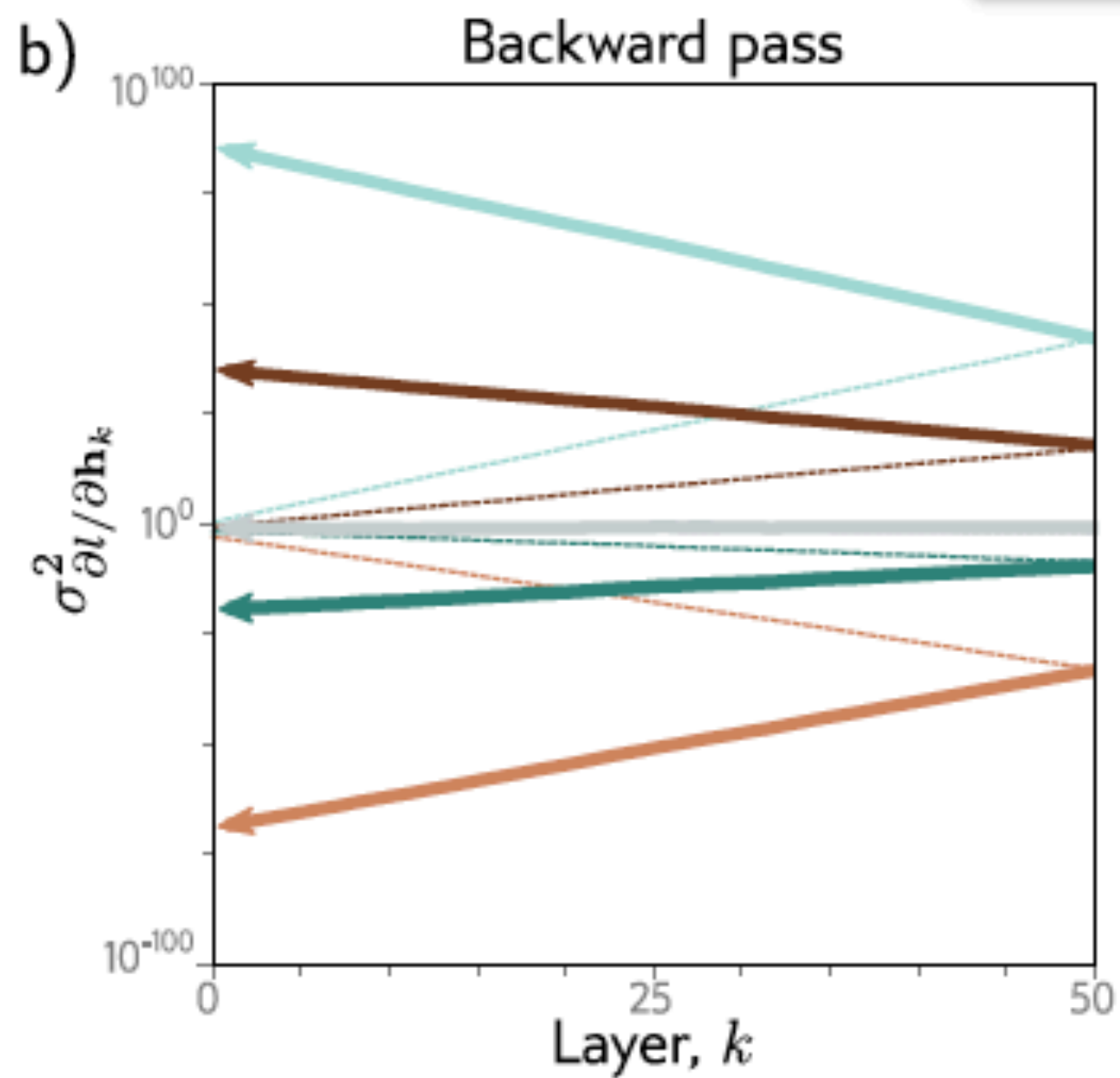
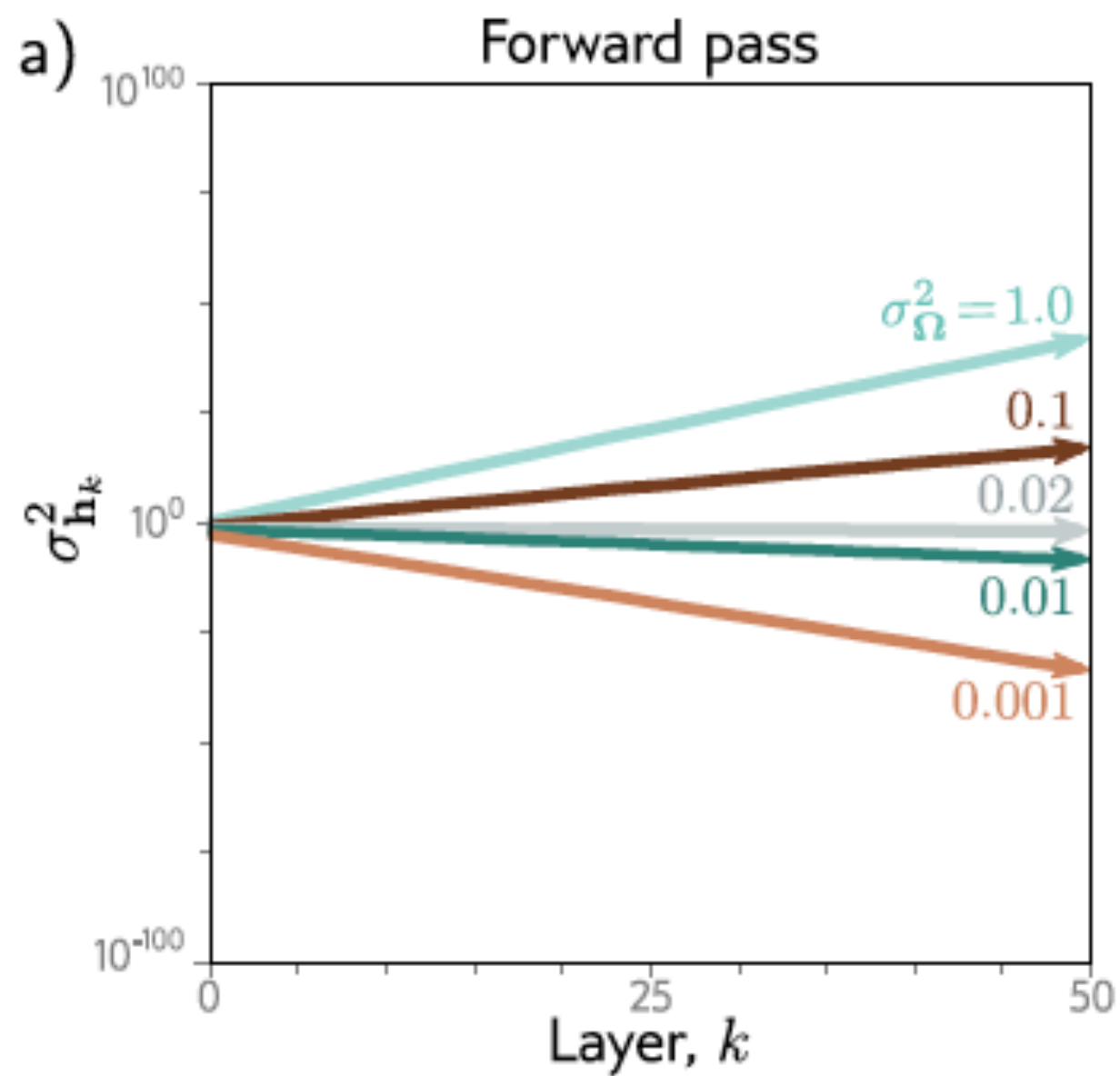


- **Hyper-parameters** tuning



Initialisation

- Crucial step to make the DNN converge
- If not done properly, we run into vanishing and exploding gradients
- General rule:
 - No zero weights (to avoid symmetry problems)
 - Controlled variance to avoid variance to compound up or down with depth
- Different initialisation strategies depending on the type of activation layer
 - HE init for Relus,
- Biases to zero, except for some architectures such as LSTMs



Regularisation

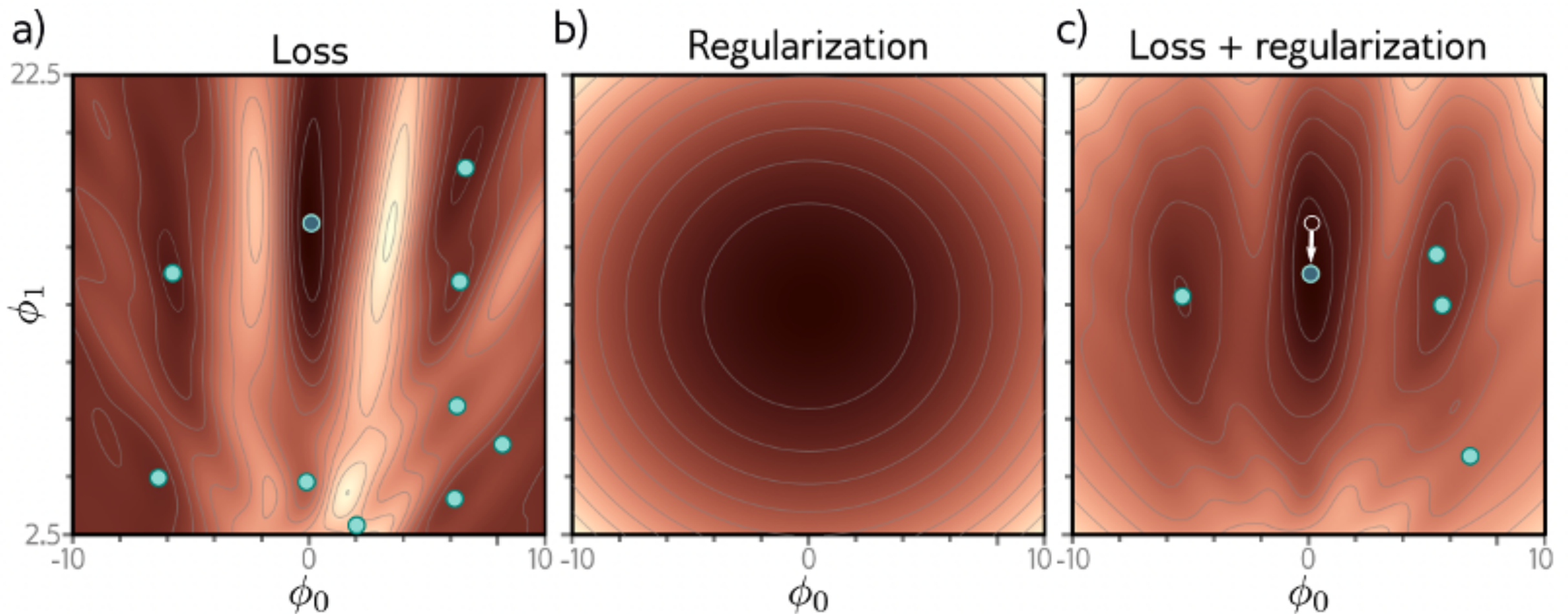
- Why is there a generalization gap between training and test data?
- Overfitting (model describes statistical peculiarities)
- Model unconstrained in areas where there are no training examples
- Regularization = methods to reduce the generalization gap
- Technically means adding terms to loss function
- But colloquially means any method (hack) to reduce gap

Explicit regularisation

- Explicit

$$L = \underbrace{\frac{1}{N} \sum_i L_i}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization loss}}$$

Two interpretations



Explicit regularisation

- Explicit

$$L = \underbrace{\frac{1}{N} \sum_i L_i}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization loss}}$$

- Most common: L2 (ridge regression)
- In NN, usually just for weights (aka weight decay)
- How to compute the regularisation gradient?

How is implemented

$$L = \underbrace{\frac{1}{N} \sum_i L_i}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization loss}}$$

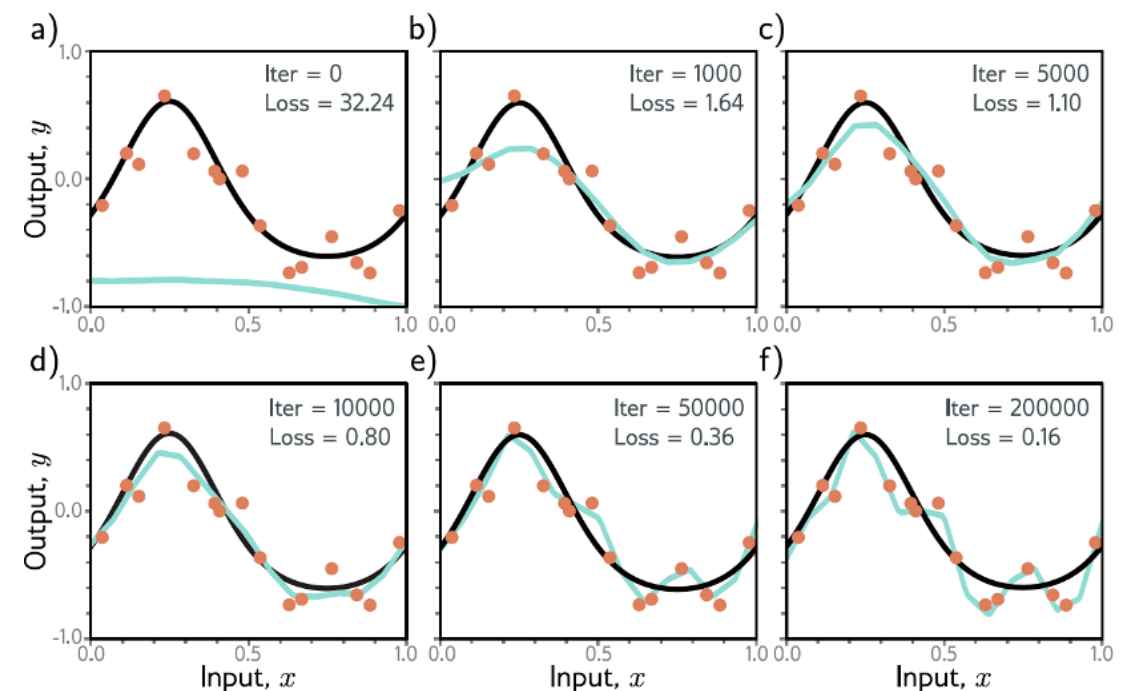
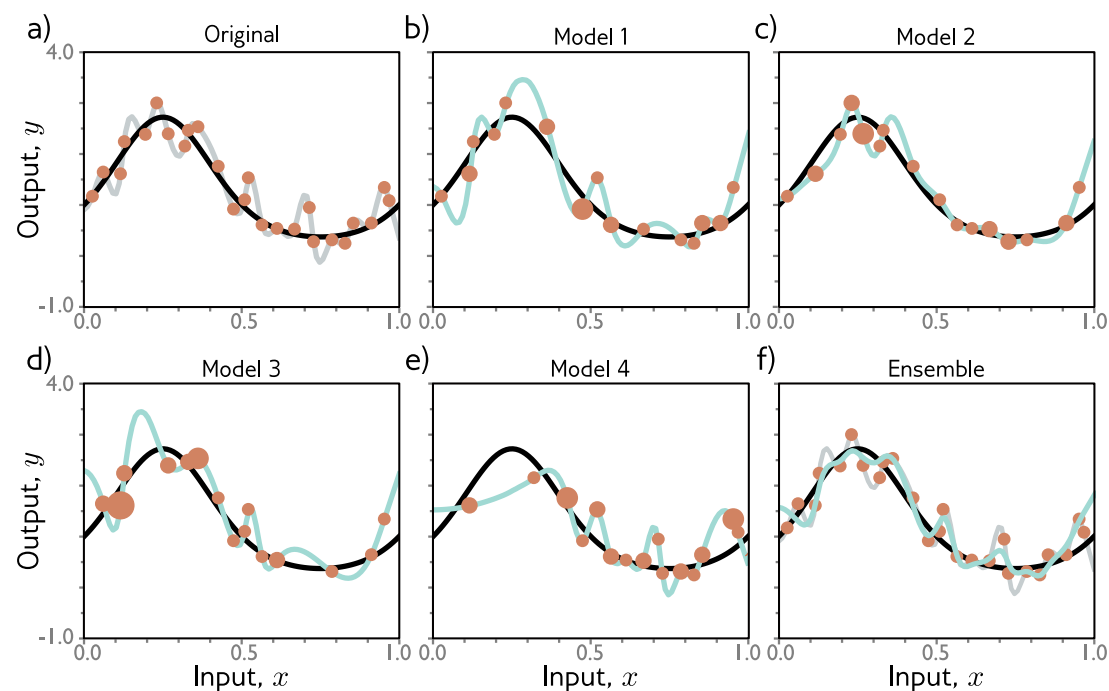
- Purely local.
- Does not depend on downstream errors or activations
- How it is implemented
 - PyTorch: out of the layer
 - Keras/tensorflow: in each layer

Implicit regularisation

- SGD has an effect on the path
 - GD disfavors areas with steep gradients
 - SGD favours places where the gradient is stable
(where all batches to have similar gradients)

Implicit regularisation

- Early stopping
 - Avoid weights getting too large and overfitting the result
 - Reduces effective model complexity
- Ensemble (Average together multiple models)
 - Take mean or median, use different initialisation and/or models
 - Add bagging (subsample subsets of data w)



Dropout

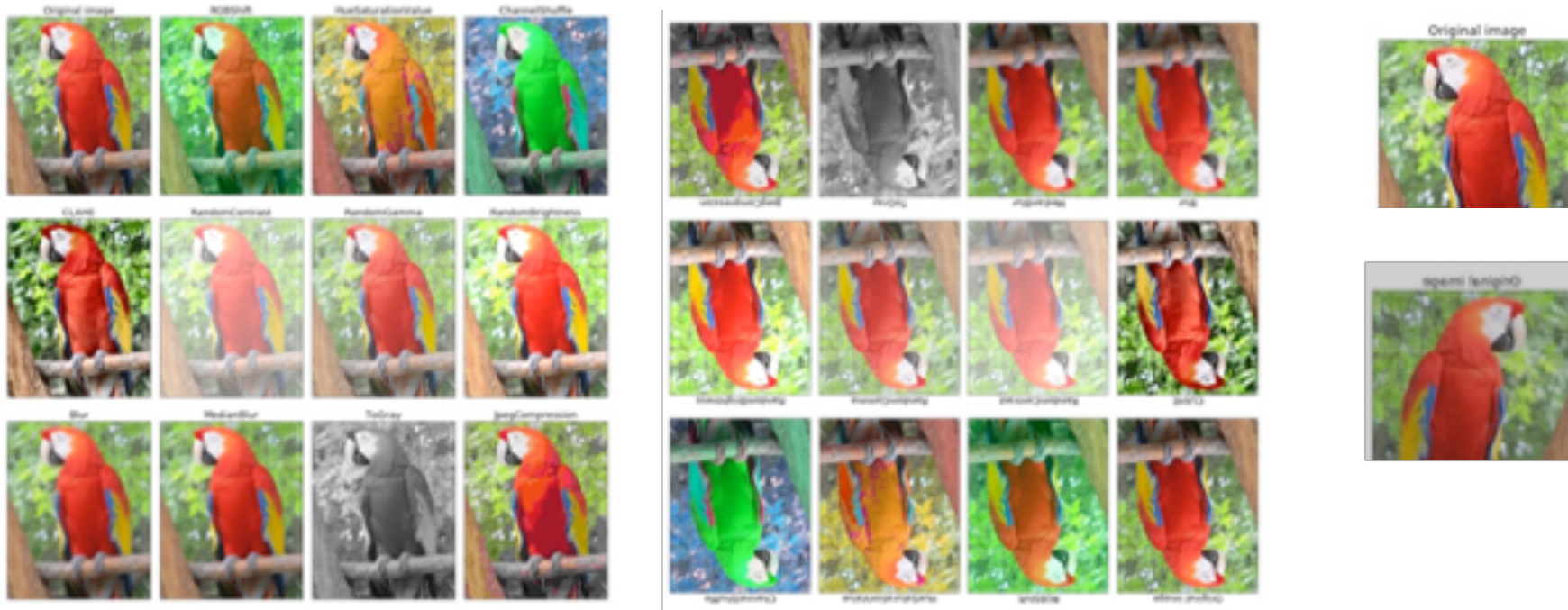
- Clamp a random subset (typically 50%) of hidden units to zero at each iteration of SGD
- Training a different network each time
- Inference
 - Full network: Weights adjusted to probability p of being active in the training (originally, in inference)
 - Ensemble (monte Carlo): run multiple inferences with dropout active. Provides mean and variance
- Helps removing non smooth predictions far away from input data

Increase data

- Increase data
 - Add noise to inputs or to labels
 - Transfer learning: retrain a model for a new task
 - Multi-task learning: learn multiple task for a single model
- Data augmentation
 - Self supervised

Regularization examples

- **Data augmentation** : generate artificial variations of data that still keep the same label



What would you do?
For cars? For faces?
For song-styles?
For OCR? ...

<https://explore.albumentations.ai/>

- **Dropout** —> train with random number of neurons “*ignored*” (this requires “re-scaling” either during training or at test time to “scale” the amount of signal accumulated if all neurons or only a % of them are active)

Make function smoother

Explicit L2
regularization

Early
stopping

Apply noise
to inputs

Apply noise
to outputs
(label smoothing)

Increase data

Data
augmentation

Multi-task
learning

Transfer
learning

Ensembling

Bayesian
approach

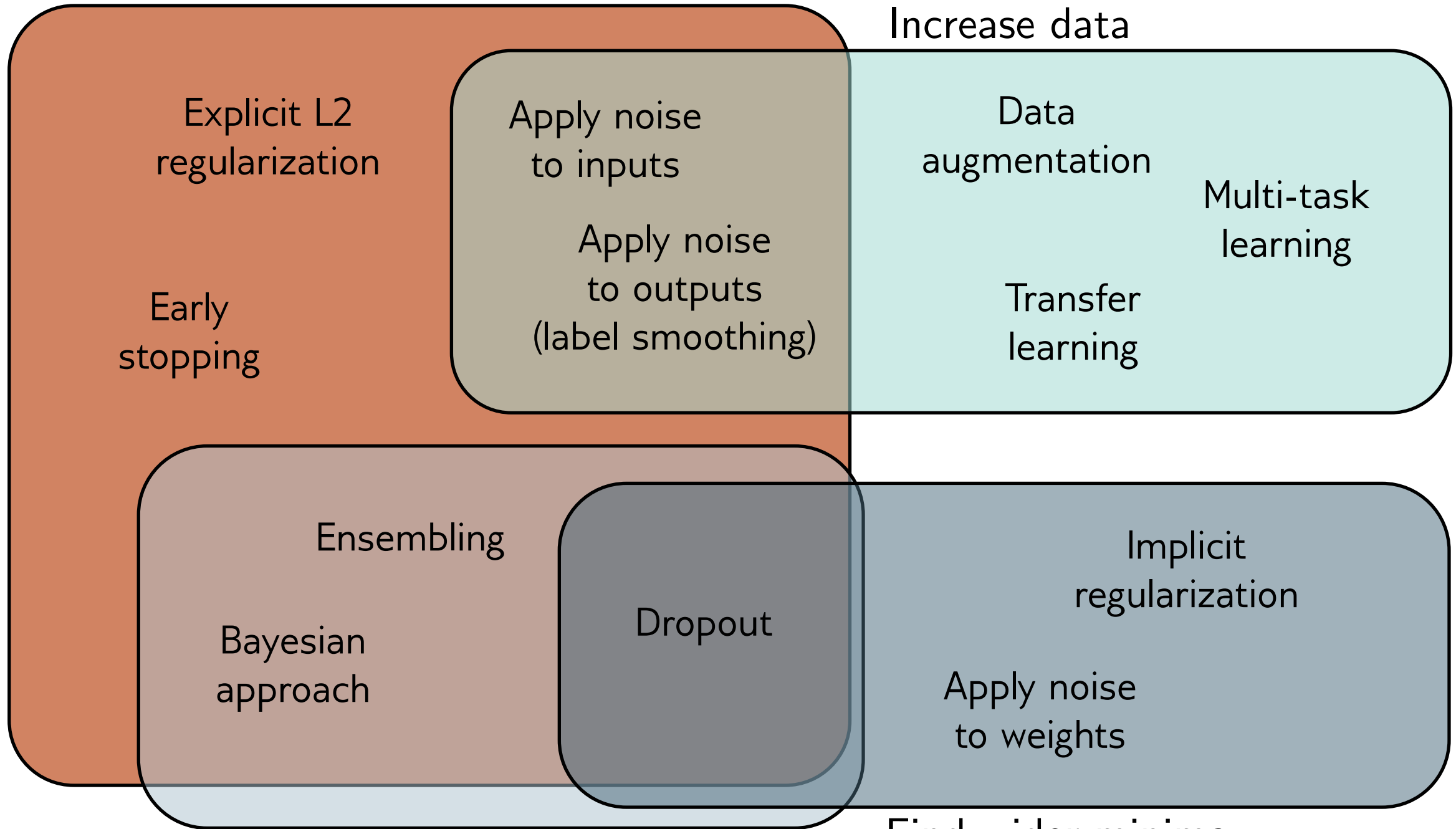
Dropout

Implicit
regularization

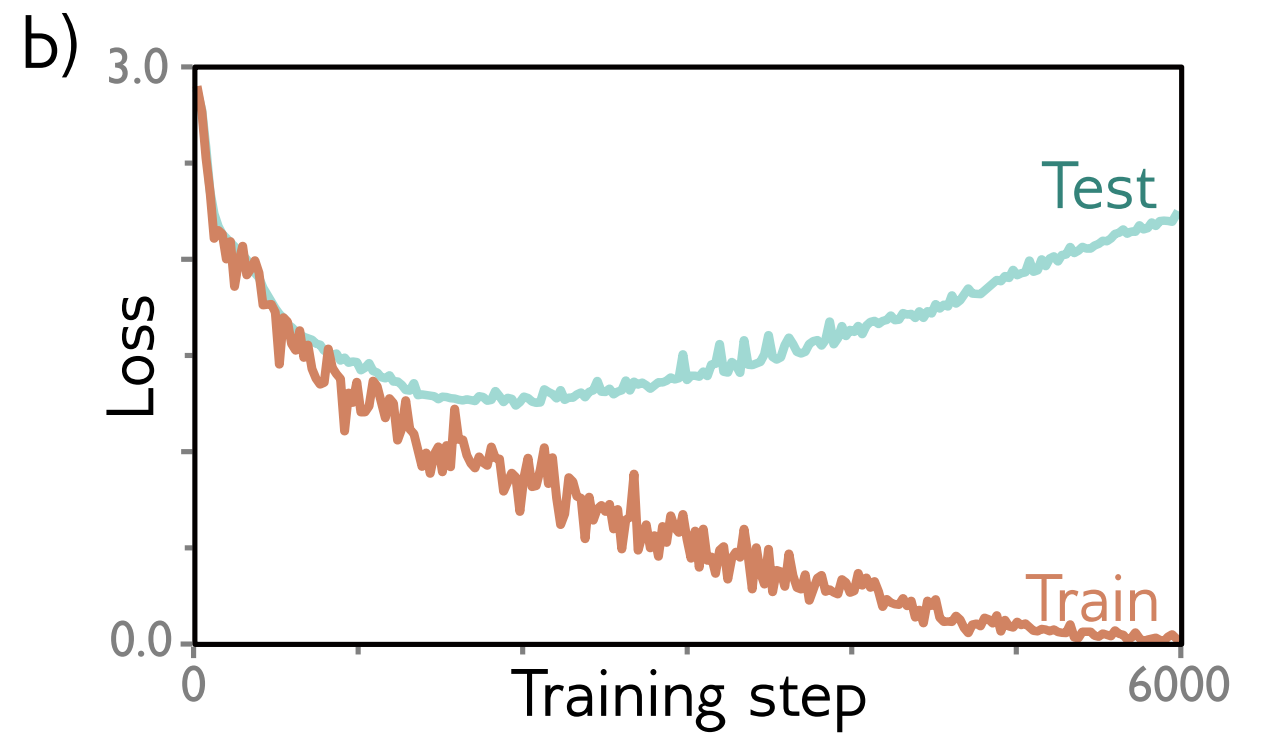
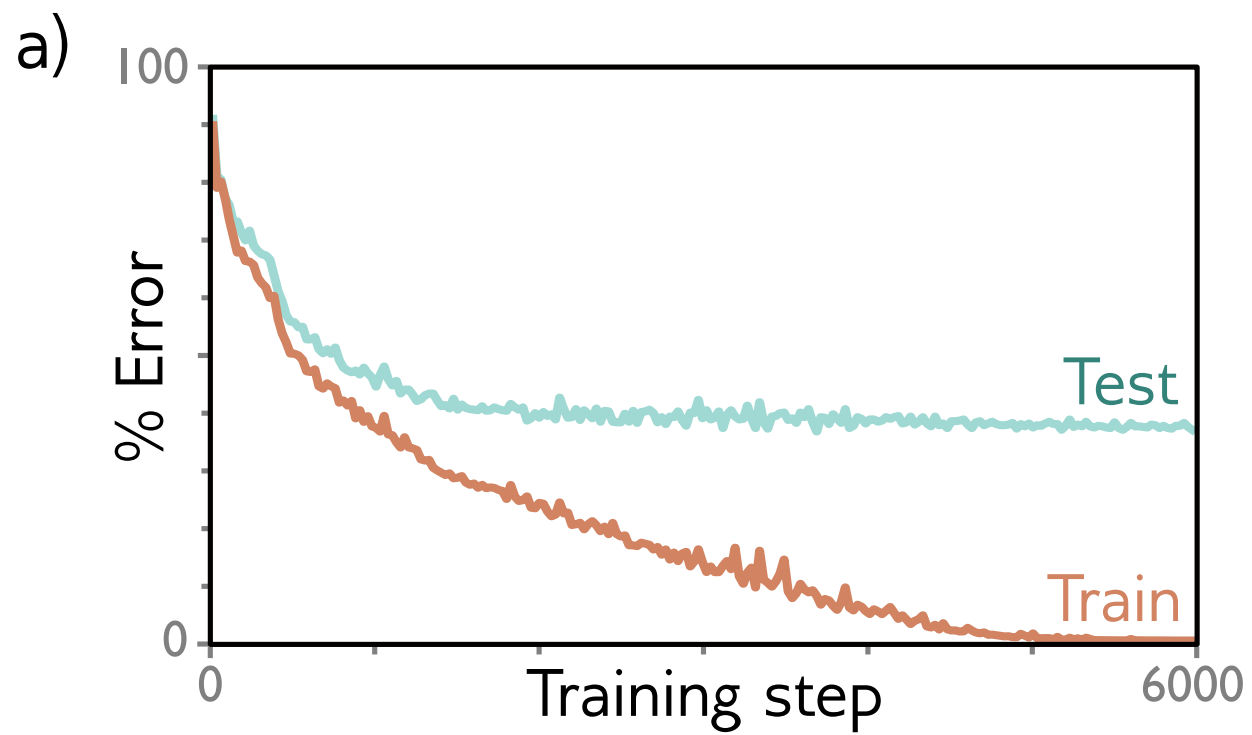
Apply noise
to weights

Combine multiple models

Find wider minima



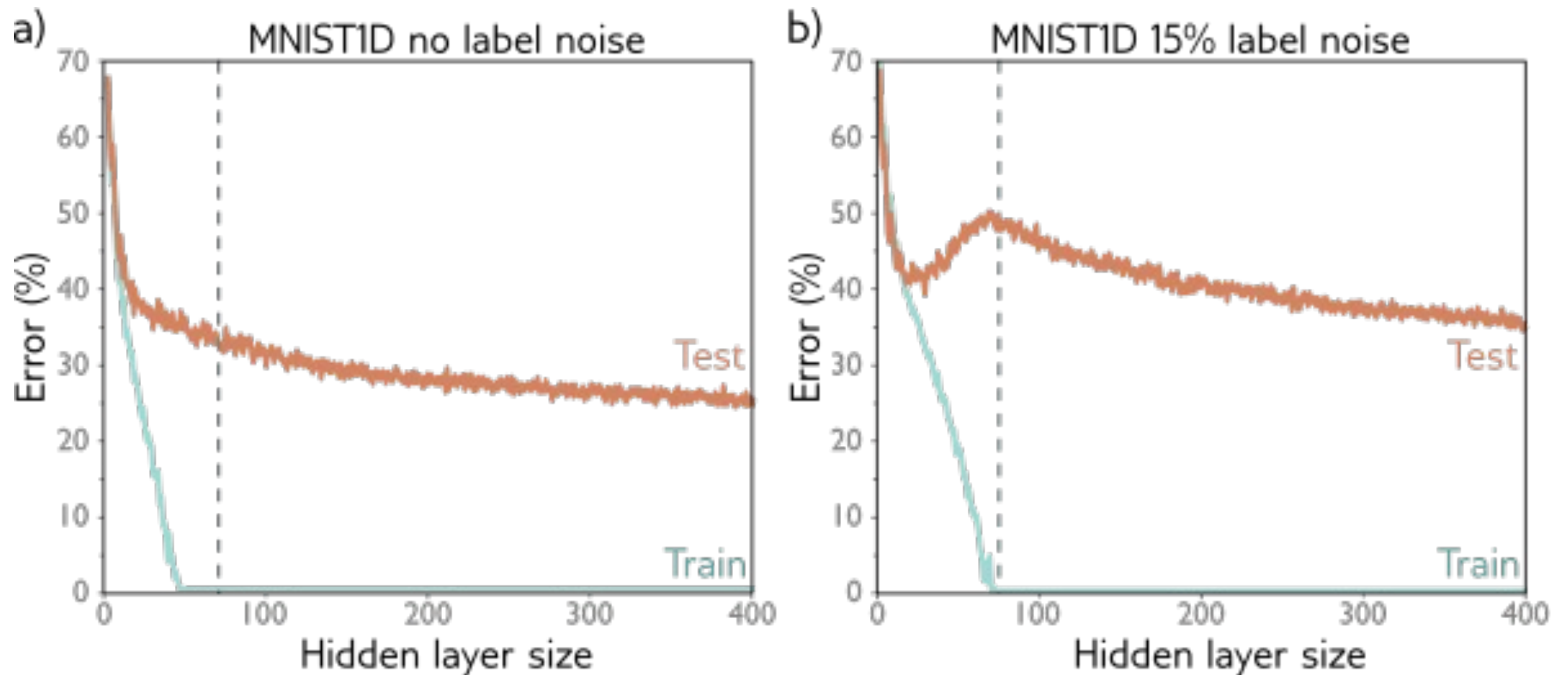
Performance



- What happens here?

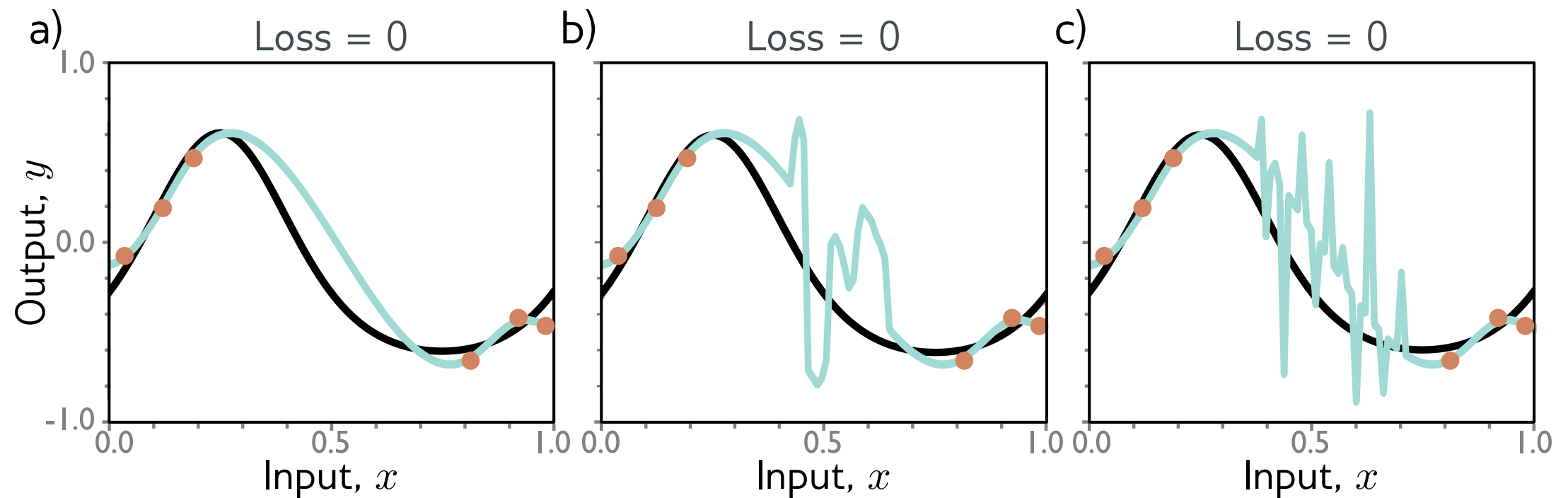
Double descent

Nakkiran et al, 2019



- What happens to the test error as the capacity of the model increases?

Double descent



- Which one has a better generalisation?
- Why should the model choose one or the other?

What about hyperparameters?

What about hyperparameters?

- Structure of the network: how much capacity in the modern regime (vs the bias/variance one)
- Regularization, learning rate ...
- Neural architecture search for the model (for constrained systems in power or in domain)
 - Define a set of models to explore
 - Search strategy (RL, evolutionary ...)
 - Many models, may require strategies for performance evaluation
- Structured exploration for other hyper parameters

Take home messages

- Back propagation intuition & how to code an new module
- Performance: changes in bias variance trade off
- Initialisation
- Regularization

- Define a neural network
- Initialize params with He initialization
- Define loss function
- Choose optimization algorithm
- Choose initial learning rate
- Choose learning rates schedule
- Make some random data
- Train for 100 batches

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from torch.optim.lr_scheduler import StepLR

# define input size, hidden layer size, output size
D_i, D_k, D_o = 10, 40, 5
# create model with two hidden layers
model = nn.Sequential(
    nn.Linear(D_i, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_o))

# He initialization of weights
def weights_init(layer_in):
    if isinstance(layer_in, nn.Linear):
        nn.init.kaiming_uniform(layer_in.weight)
        layer_in.bias.data.fill_(0.0)
model.apply(weights_init)

# choose least squares loss function
criterion = nn.MSELoss()
# construct SGD optimizer and initialize learning rate and momentum
optimizer = torch.optim.SGD(model.parameters(), lr = 0.01, momentum=0.9)
# object that decreases learning rate by half every 10 epochs
scheduler = StepLR(optimizer, step_size=10, gamma=0.5)

# create 100 dummy data points and store in data loader class
x = torch.randn(100, D_i)
y = torch.randn(100, D_o)
data_loader = DataLoader(TensorDataset(x,y), batch_size=10, shuffle=True)

# loop over the dataset 100 times
for epoch in range(100):
    epoch_loss = 0.0
    # loop over batches
    for i, data in enumerate(data_loader):
        # retrieve inputs and labels for this batch
        x_batch, y_batch = data
        # zero the parameter gradients
        optimizer.zero_grad()
        # forward pass
        pred = model(x_batch)
        loss = criterion(pred, y_batch)
        # backward pass
        loss.backward()
        # SGD update
        optimizer.step()
        # update statistics
        epoch_loss += loss.item()
    # print error
    print(f'Epoch {epoch:5d}, loss {epoch_loss:.3f}')
    # tell scheduler to consider updating learning rate
    scheduler.step()
```

Bibliography - Resources for materials in this block

- Stanford online materials on Deep learning for Computer Vision (<http://cs231n.stanford.edu>) and Deep Learning (<https://cs230.stanford.edu/>)
- Ian Goodfellow, Yoshua Bengio, Aaron Courville, **Deep Learning**, MIT Press, 2016.
<http://www.deeplearningbook.org>
- Understanding deep learning, Simon J.D. Prince, MIT Press, <https://udlbook.github.io/udlbook/>

Backprop details

- We need matrix computations

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} \quad \frac{\partial f}{\partial \mathbf{a}} = \begin{bmatrix} \frac{\partial f}{\partial a_1} \\ \frac{\partial f}{\partial a_2} \\ \frac{\partial f}{\partial a_3} \\ \frac{\partial f}{\partial a_4} \end{bmatrix} \quad \mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix} \quad \frac{\partial f}{\partial \mathbf{A}} = \begin{bmatrix} \frac{\partial f}{\partial a_{11}} & \frac{\partial f}{\partial a_{12}} & \frac{\partial f}{\partial a_{13}} \\ \frac{\partial f}{\partial a_{21}} & \frac{\partial f}{\partial a_{22}} & \frac{\partial f}{\partial a_{23}} \\ \frac{\partial f}{\partial a_{31}} & \frac{\partial f}{\partial a_{32}} & \frac{\partial f}{\partial a_{33}} \\ \frac{\partial f}{\partial a_{41}} & \frac{\partial f}{\partial a_{42}} & \frac{\partial f}{\partial a_{43}} \end{bmatrix}$$

$$\mathbf{f} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix} \quad \mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} \quad \frac{\partial \mathbf{f}}{\partial \mathbf{a}} = \begin{bmatrix} \frac{\partial f_1}{\partial a_1} & \frac{\partial f_2}{\partial a_1} & \frac{\partial f_3}{\partial a_1} \\ \frac{\partial f_1}{\partial a_2} & \frac{\partial f_2}{\partial a_2} & \frac{\partial f_3}{\partial a_2} \\ \frac{\partial f_1}{\partial a_3} & \frac{\partial f_2}{\partial a_3} & \frac{\partial f_3}{\partial a_3} \\ \frac{\partial f_1}{\partial a_4} & \frac{\partial f_2}{\partial a_4} & \frac{\partial f_3}{\partial a_4} \end{bmatrix}$$

Scalar and matrix derivatives

- Scalar

$$f_3 = \beta_3 + \omega_3 h_3$$

$$\frac{\partial f_3}{\partial \beta_3} = \frac{\partial}{\partial \omega_3} \beta_3 + \omega_3 h_3 = 1$$

$$\frac{\partial f_3}{\partial h_3} = \frac{\partial}{\partial h_3} (\beta_3 + \omega_3 h_3) = \omega_3$$

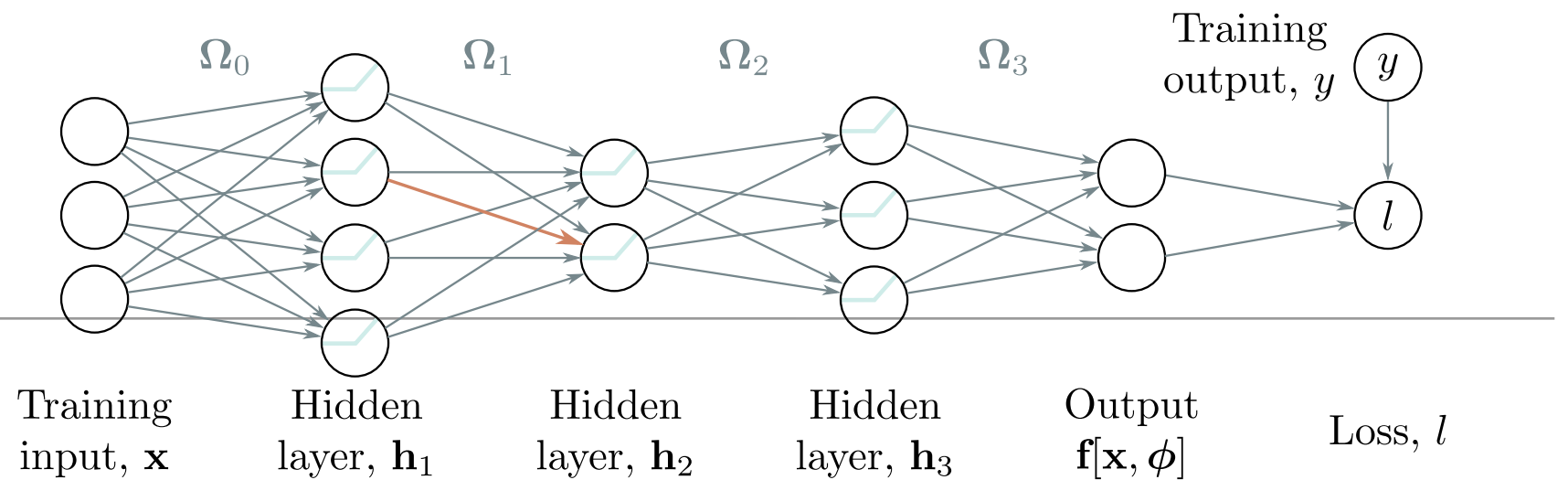
- Matrix

$$\mathbf{f}_3 = \boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3$$

$$\frac{\partial \mathbf{f}_3}{\partial \boldsymbol{\beta}_3} = \frac{\partial}{\partial \boldsymbol{\beta}_3} (\boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3) = \mathbf{I}$$

$$\frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} = \frac{\partial}{\partial \mathbf{h}_3} (\boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3) = \boldsymbol{\Omega}_3^T$$

Forward pass



- Compute functions forward for each layer

$$\mathbf{f}_0 = \beta_0 + \mathbf{\Omega}_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \mathbf{\Omega}_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

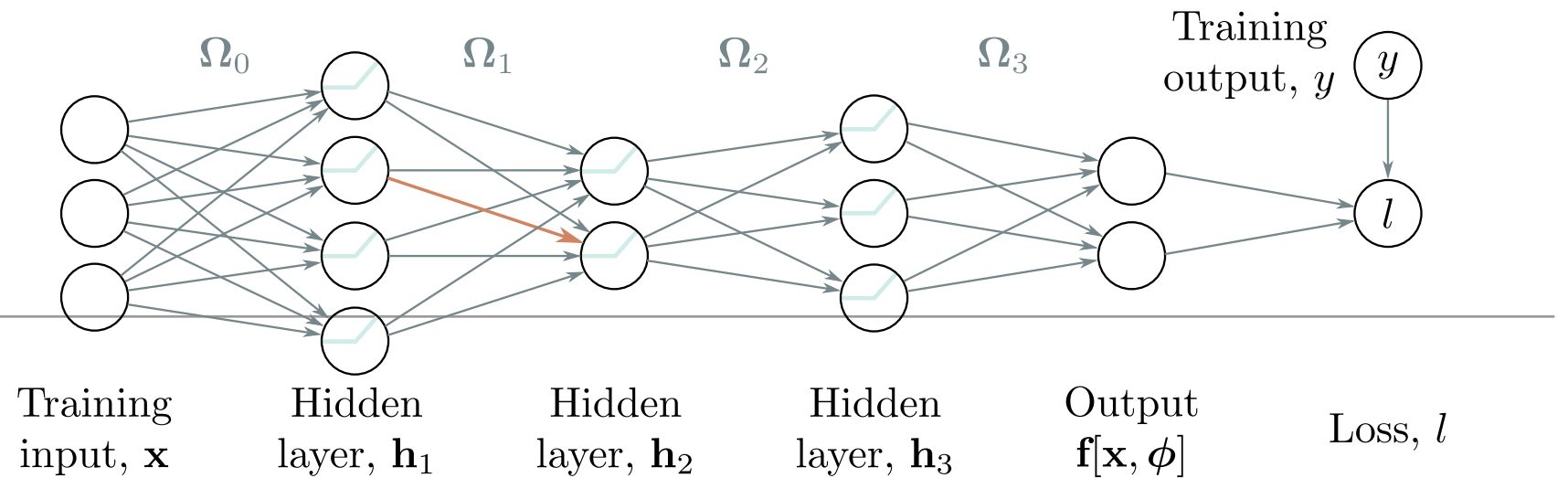
$$\mathbf{f}_2 = \beta_2 + \mathbf{\Omega}_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \mathbf{\Omega}_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

Backward pass



- Take derivatives of output w.r.t. intermediate functions

$$\frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

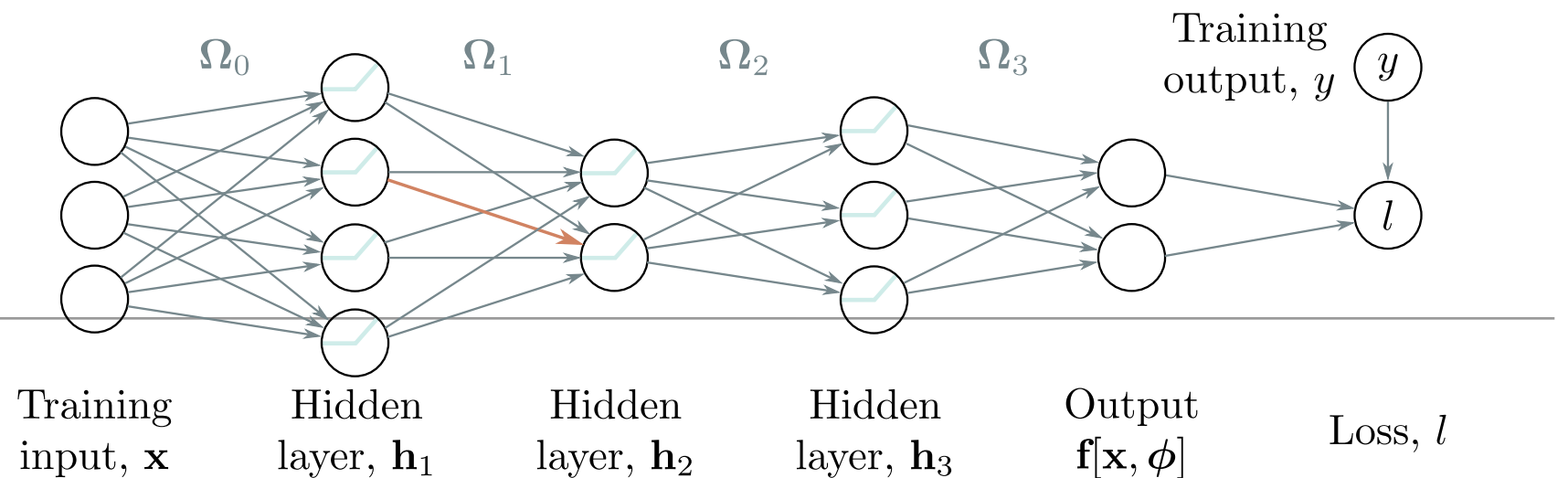
$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

Backward pass



- Take derivatives of output w.r.t. intermediate functions

$$\frac{\partial \ell_i}{\partial \mathbf{f}_3} \quad \begin{matrix} \nearrow ?? \\ \nearrow \end{matrix} \quad \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} = \frac{\partial}{\partial \mathbf{h}_3} (\beta_3 + \Omega_3 \mathbf{h}_3) = \Omega_3^T$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

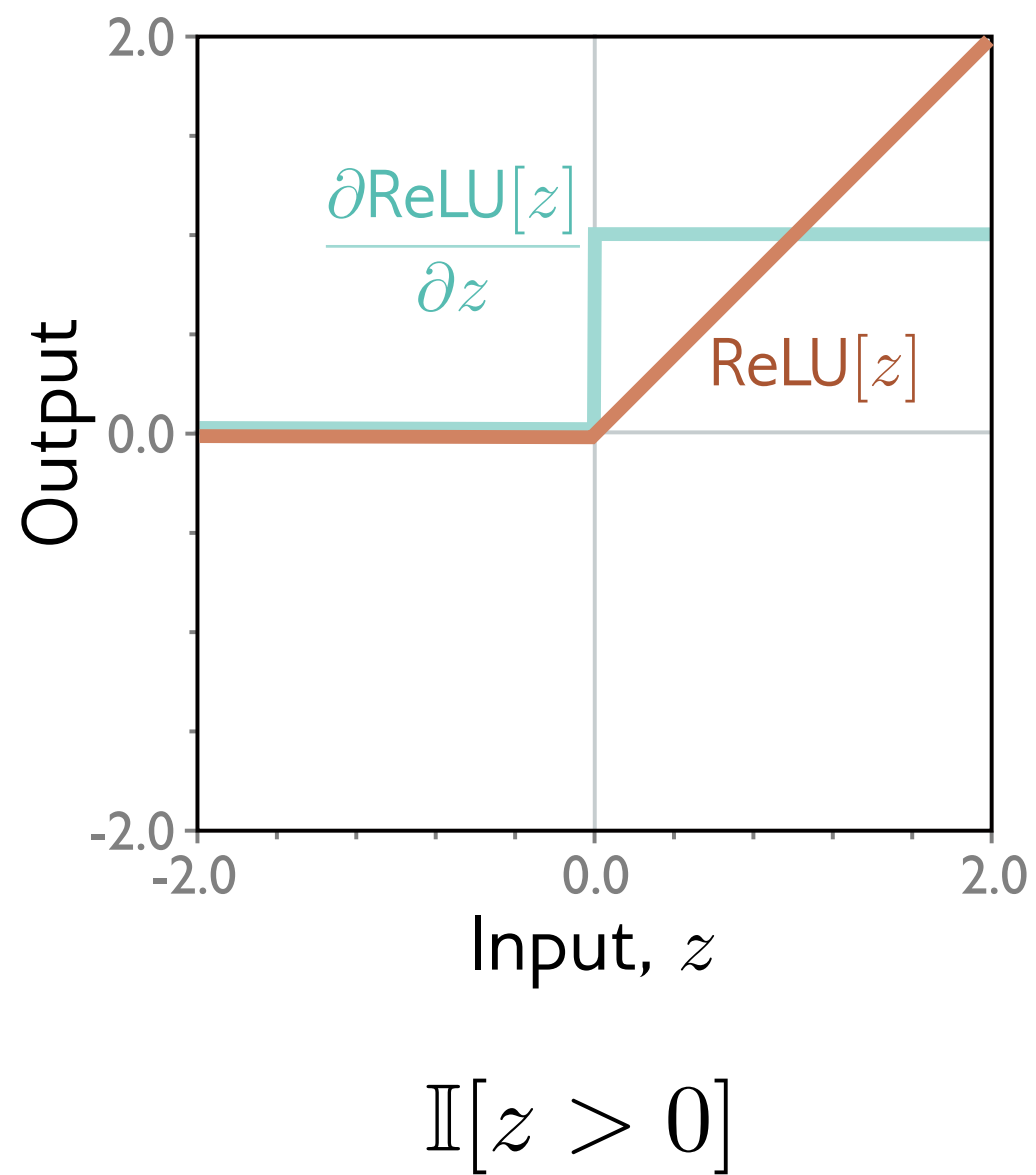
$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

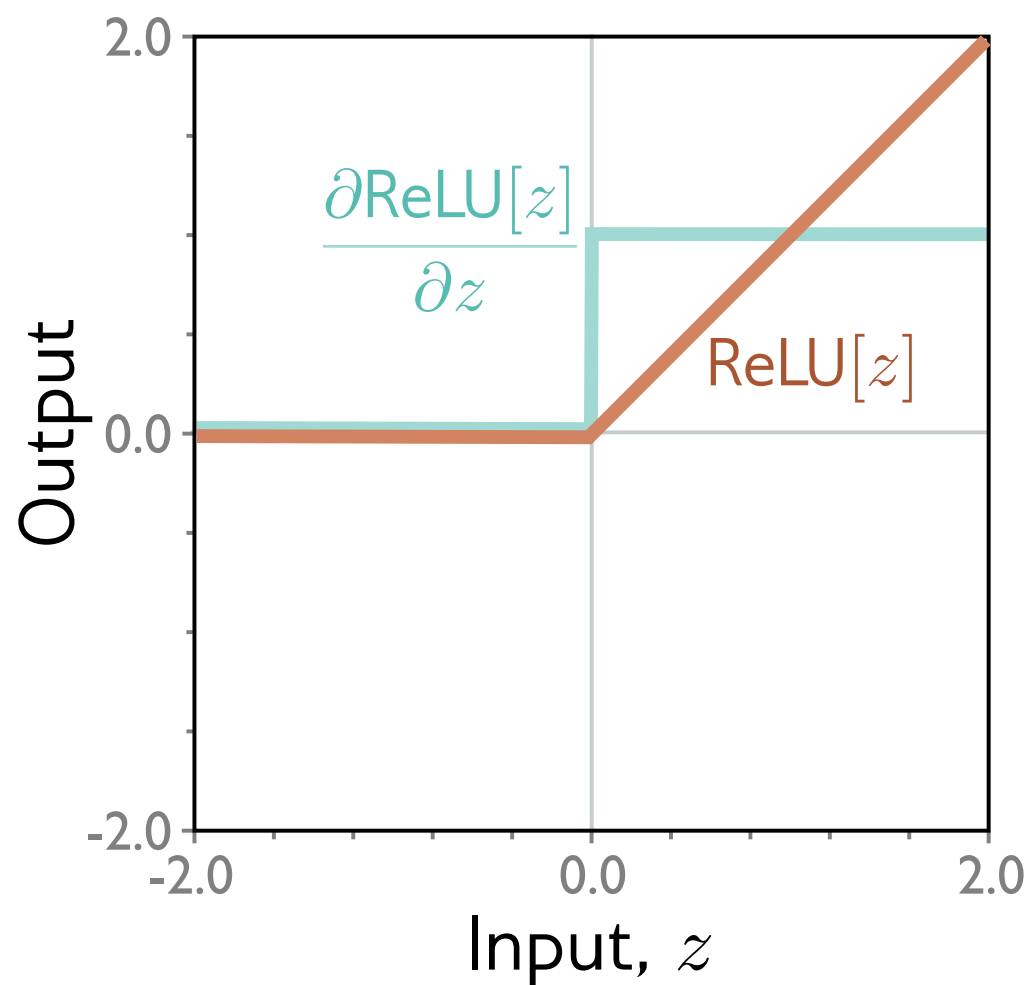
Derivative of Relu

- Single function



Derivative of Relu

- Single function



$$\mathbb{I}[z > 0]$$

- Matrix form

$$\mathbf{a} = \text{ReLU}[\mathbf{b}]$$

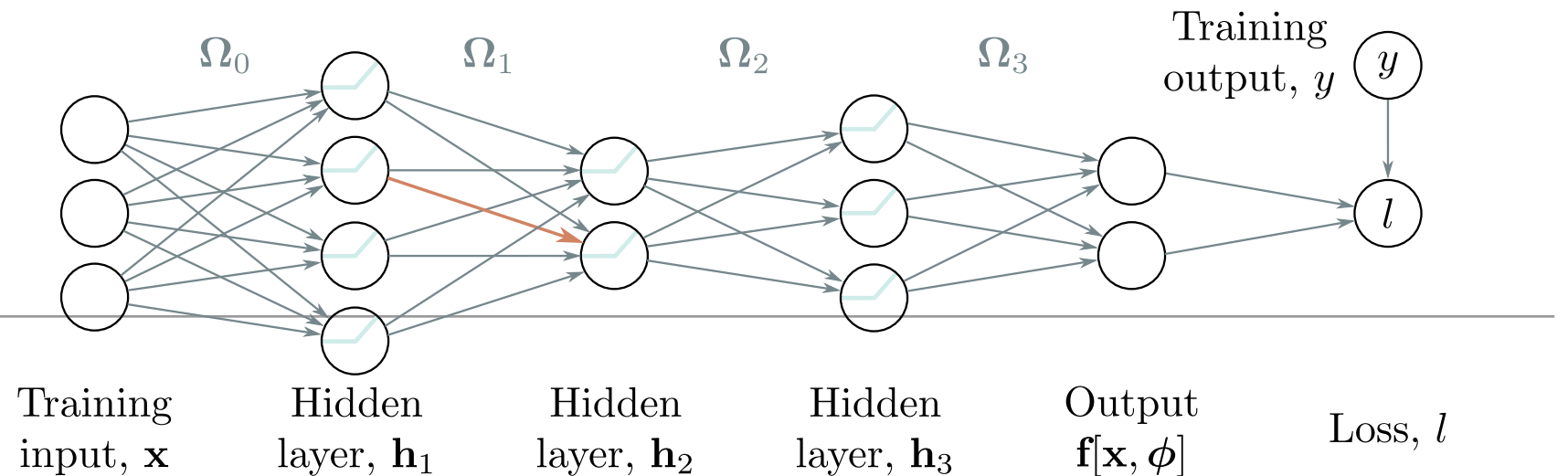
- For $\mathbf{b}$ in $\mathbb{R}^3$

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \text{ReLU}[b_1] \\ \text{ReLU}[b_2] \\ \text{ReLU}[b_3] \end{bmatrix} \quad \frac{\partial \mathbf{a}}{\partial \mathbf{b}} = \begin{bmatrix} \mathbb{I}[b_1 > 0] & 0 & 0 \\ 0 & \mathbb{I}[b_2 > 0] & 0 \\ 0 & 0 & \mathbb{I}[b_3 > 0] \end{bmatrix}$$

- Also pointwise multiplication

$$\mathbb{I}[\mathbf{b} > 0] \odot$$

Backward pass



- Take derivatives of output w.r.t. intermediate functions

$$\frac{\partial \ell_i}{\partial \mathbf{f}_3} \xrightarrow{\mathbb{I}[\mathbf{f}_2 > 0]} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} = \frac{\partial}{\partial \mathbf{h}_3} (\beta_3 + \Omega_3 \mathbf{h}_3) = \Omega_3^T$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

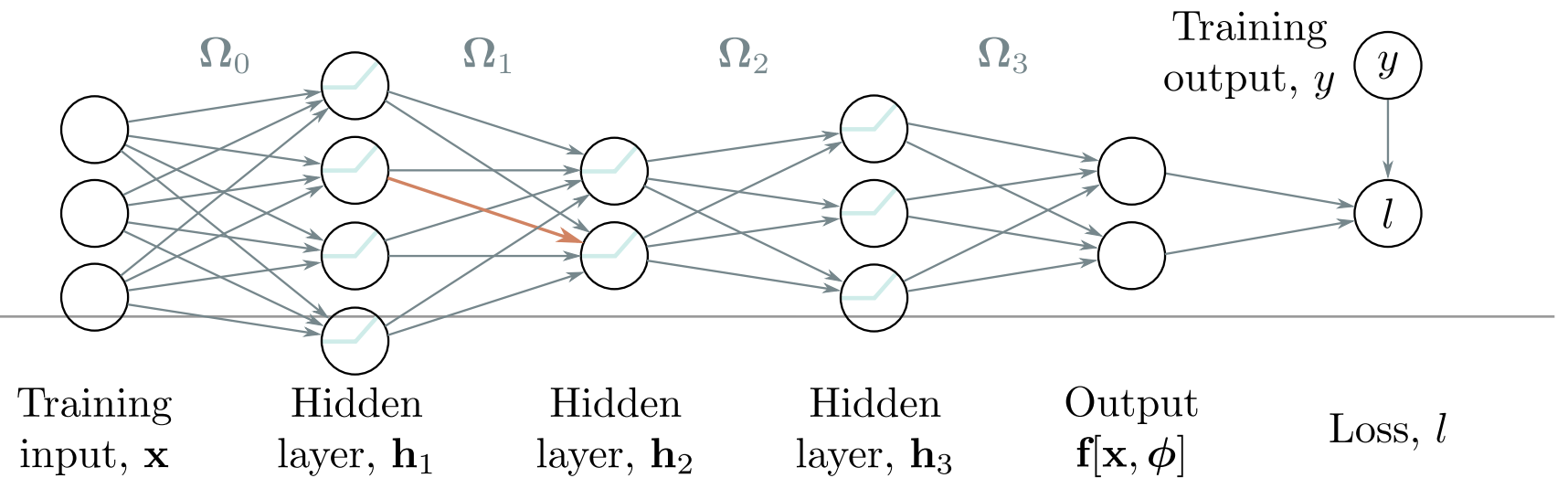
$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

Backward pass



- Take derivatives w.r.t. parameters (weights and biases)

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \Omega_k} = \frac{\partial \mathbf{f}_k}{\partial \Omega_k} \frac{\partial \ell_i}{\partial \mathbf{f}_k}$$

$$= \frac{\partial}{\partial \Omega_k} (\beta_k + \Omega_k \mathbf{h}_k) \frac{\partial \ell_i}{\partial \mathbf{f}_k}$$

$$= \frac{\partial \ell_i}{\partial \mathbf{f}_k} \mathbf{h}_k^T$$

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial \mathbf{f}_k}{\partial \beta_k} \frac{\partial \ell_i}{\partial \mathbf{f}_k}$$

$$= \frac{\partial}{\partial \beta_k} (\beta_k + \Omega_k \mathbf{h}_k) \frac{\partial \ell_i}{\partial \mathbf{f}_k}$$

$$= \frac{\partial \ell_i}{\partial \mathbf{f}_k},$$

Pros and cons of back-propagation

- Extremely efficient
 - Only need matrix multiplication (and thresholding for ReLU functions)
- Memory hungry – must store all the intermediate quantities
- Sequential
 - can process multiple batches in parallel
 - but things get harder if the whole model doesn't fit on one machine.